REOAGMS: Presentation script

This script gives the **exact words** each member says. **Part A** is the 5-minute live demo (one continuous demo on one laptop, hard time limits). **Part B** is the full word-for-word narration of each member's seven slides in the team deck `REOAGMS-Team-Presentation.pptx` (the same text is in the slides' speaker notes).

Before you start

- Stack running (`docker compose up -d`), browser signed in as `admin@reoagms.com` on http://localhost:5173, a second tab on Grafana. Zoom 100 %, notifications silenced.
- Two minutes earlier, open the Alerts page and resolve the open alerts; the simulator then raises fresh events within about a minute.
- Rehearse with a stopwatch. Hand over in one sentence, as written.

Credentials

Use	Login
Console (all roles)	`admin@reoagms.com` , `manager@reoagms.com` , `operator@reoagms.com` , `engineer@reoagms.com` , `executive@reoagms.com` · password `Reoagms@2026`
Grafana http://localhost:3001	user `admin` · password `reoagms`
RabbitMQ http://localhost:15672	`reoagms` / `reoagms_password`
Prometheus http://localhost:9090	no login

Speaking order and time (Part A)

Time	Member
0:00–0:40	Charith
0:40–1:10	Nemal
1:10–2:00	Govindu
2:00–2:40	Thurunu
2:40–3:10	Yasiru
3:10–4:00	Sachinthana
4:00–5:00	Charith, then all members

If a fault has not arrived by 1:10, Govindu says "the simulator injects one every minute" and Thurunu uses the newest order already on the board.

Part A: the 5-minute live demo, exact words

0:00–0:40 · Charith

On screen: Login page, then Control room

Charith says: Good morning. We are presenting REOAGMS, the Renewable Energy Operations, Asset and Grid Management System. It is a set of Spring Boot microservices behind a single API gateway, connected by RabbitMQ events, and everything you will see is real data in six PostgreSQL databases. I sign in as the administrator; the gateway checks my token and my role. This is the control room: live generation, open alerts, device health, an hourly energy curve and a live alert feed. Nemal will show where this data comes from.

0:40–1:10 · Nemal

On screen: Assets page; open Add, then cancel

Nemal says: Thank you, Charith. The Asset Service owns the registry: five facilities, eleven assets and thirty-two sensors, covering solar, wind, hydro, battery and tidal. Lists are paginated and searchable on the server, and every form is validated, so a blank name or a future installation date is rejected. Only administrators and operations managers can add or delete; other roles see this page read-only. Every other service refers to these assets by ID. Govindu will show them producing live data.

1:10–2:00 · Govindu

On screen: Live monitoring page; wait for the toast; open the Alerts page if needed

Govindu says: Thanks, Nemal. The Monitoring Service receives a reading from every sensor every few seconds and checks it against our alert rules. This tile is a live sensor, and the dashed line is its threshold. The simulator has just injected a fault, and here is the alert, instantly. The rule fired, the alert was stored once, one event was published to RabbitMQ, and the Notification Service pushed it to this browser over WebSocket. We keep one open alert per rule and asset, so operators are never flooded. That same event also reached Thurunu's service.

2:00–2:40 · Thurunu

On screen: Maintenance board: newest card, Assign, Start, Complete

Thurunu says: Yes. The Maintenance Service consumed the same event and, because the severity is high, it opened a corrective work order automatically. Here it is on the board, in Created. I assign a technician, start the work, and complete it. Each step is its own endpoint, and the finished job is written to the maintenance history. So a fault becomes tracked work with no manual step. Yasiru will show how people are told about it.

2:40–3:10 · Yasiru

On screen: Notifications page

Yasiru says: The Notification Service consumed the same alert event from its own RabbitMQ queue. In this history you can see every message: the in-app push was delivered instantly, and every email attempt is stored with its status, retry count and reason. The scheduler retries up to three times, and I can retry any message from

here. Messages that keep failing go to a dead-letter queue, so nothing is lost. Sachinthana will turn all this data into reports.

3:10–4:00 · Sachinthana

On screen: Analytics page: pick facility; Generate → Energy report; click the download icon

Sachinthana says: The Analytics Service calls the other services over REST, never their databases. For this facility it integrates average power into kilowatt-hours per hour, and calculates availability, efficiency, capacity factor and utilisation. The chart is hourly energy. I generate an energy report for the last day and export it as CSV for management. We also keep KPIs, dashboards and performance reports, and executives see all of this read-only. Charith will close.

4:00–5:00 · Charith

On screen: Sign out, sign in as executive@reoagms.com, then the Grafana tab

Charith says: To close. On security: I sign in as the executive. There are no action buttons, and the API itself returns 403 for any write. For operations, Grafana shows platform health from Prometheus, which scrapes all seven services. On quality: sixty test methods, validation and error handling in every service, and a versioned event contract that keeps the services compatible. The whole system starts with one docker compose command. Thank you. We are happy to take questions.

4:50–5:00 · All members

All say: Thank you.

Part B: full narration of the team deck, exact words per member

Charith

Slide 1. Good morning. We are presenting REOAGMS, the Renewable Energy Operations, Asset and Grid Management System, built for EC8208 Software Architecture. I will start with the architecture, the identity service, the gateway and the infrastructure, and then each teammate will present their own service.

Slide 2. My scope covers the architecture decision, the Identity Service, the API Gateway, the React console, Docker, RabbitMQ, Kubernetes and the final integration. Together these satisfy the authentication, role-based access, audit and gateway requirements: FR-01, FR-02, FR-03, FR-16 and FR-18.

Slide 3. This is the path of every request. The browser is served by nginx and talks only to the API Gateway. The gateway validates the JWT and the user role, then routes the call to the Identity Service, which owns its own PostgreSQL database.

Slide 4. In the Identity Service, tokens are signed with HS256 and carry the user role. Passwords are hashed with BCrypt, and self-registration always creates the least privileged role. The gateway has seven routes, answers 401 without a token and 403 when the role is not allowed. Logins and user changes are written to an audit log, and the whole system runs from Docker Compose with Kubernetes manifests alongside.

Slide 5. The Identity Service exposes login, registration, users and the audit log. Around it sit the shared building blocks: a RabbitMQ topic exchange with a dead-letter queue, a Redis cache, Prometheus and Grafana for monitoring, and nginx serving the production build of the console.

Slide 6. These numbers are measured from the running system: nineteen containers in Docker Compose, five seeded roles, and seven gateway routes. We verified the role matrix: a manager can create a facility, while an operator, an engineer and an executive receive a 403.

Slide 7. In the demo I sign in as the administrator, show the control room, and open the audit log. The takeaway: one entry point and one security policy, services that stay independent and are connected by events, and a system that starts with a single command. Now Nemal will present the Asset Service.

Nemal

Slide 1. Thank you, Charith. I am Nemal, and I built the Asset Service, which is the registry for facilities, assets and sensors.

Slide 2. My scope is facility, asset and sensor management, image upload for assets, and search, filtering, pagination and sorting, with validation and unit tests. This covers requirements FR-04, FR-05 and FR-06.

Slide 3. The service is a classic layered design. The controller receives the REST call and validates it, the service applies the business rules, the repository uses Spring Data JPA, and the data is stored in the asset database.

Slide 4. A facility has one of five types, a capacity in megawatts and coordinates. An asset has one of seven types and five statuses, a rated capacity and a serial number. A sensor has one of nine metric types, a unit and optional minimum and maximum thresholds. Images are uploaded per asset, and search uses JPA specifications by name, status and type with paging.

Slide 5. Other services refer to our assets only by UUID. Monitoring uses sensor and asset identifiers, Maintenance attaches work orders to an asset, and Analytics reads the rated capacity to compute expected energy.

Slide 6. In the database there are five facilities, eleven assets and thirty-two sensors. The service has nineteen unit test methods, and validation rejects blank names, future installation dates and non-positive capacity with field-level messages.

Slide 7. In the demo I open the Assets page, switch between tabs, search, and open the add form to show validation. The takeaway: the registry is the single source of truth for identifiers, pagination keeps lists fast, and write access is limited by role. Next, Govindu will show these assets sending live data.

Govindu

Slide 1. Thank you, Nemal. I am Govindu, and I built the Monitoring Service, which turns sensor data into alerts.

Slide 2. My scope is sensor readings, device telemetry, alert rules, alert generation, historical data retrieval, the dashboard APIs and the RabbitMQ alert publisher. This covers FR-07, FR-08, FR-09, FR-10, FR-13 and FR-15.

Slide 3. This is the flow from a reading to an alert. A sensor posts a reading, the rule evaluator compares it with each applicable rule, an alert is created with a lifecycle of open, acknowledged and resolved, and an event is

published to RabbitMQ.

Slide 4. Rules support five comparison operators and four severities, and can apply to one asset or to all assets. We keep only one open alert per rule and asset, so a sustained fault does not flood operators. History can be filtered by facility, asset, sensor, metric and time window. The dashboard summary counts alerts and devices, and it is cached in Redis for ten seconds.

Slide 5. The service publishes a versioned alert event with the alert, asset, severity, value and threshold. Notification and Maintenance both consume it. If Redis is unavailable, the cache is skipped and PostgreSQL answers, so the dashboard never fails.

Slide 6. The monitoring database holds more than sixteen thousand readings and seven detection rules, and the service has eleven unit test methods. Device health uses the latest heartbeat of each asset, and dashboard answers are cached.

Slide 7. In the demo I show live sensor tiles with a threshold line, and then an alert arrives instantly. The takeaway: rules are data rather than code, events are versioned contracts, and de-duplication protects operators. Thurunu will show what happens next.

Thurunu

Slide 1. Thank you, Govindu. I am Thurunu, and I built the Maintenance Service, which turns alerts into tracked work.

Slide 2. My scope is work orders, maintenance tasks, technician assignment, preventive and corrective maintenance, maintenance history and reports, and the automatic creation of work orders from alerts. This covers FR-11 and FR-12.

Slide 3. This is the lifecycle. An alert event of high or critical severity arrives, a work order is created, a technician is assigned and starts the work, and the order is completed and written to the history.

Slide 4. A work order moves from created to assigned, in progress and completed, or it can be cancelled. Tasks have their own start and complete steps. Technicians have a specialisation and an availability. Orders are either preventive or corrective, the history keeps a record per asset, and a summary endpoint feeds the reports.

Slide 5. The service listens on its own RabbitMQ queue for the alert event. It creates a corrective order for high and critical alerts, it skips an asset that already has an active corrective order, and it stores the alert identifier so a redelivered message never creates a duplicate.

Slide 6. The database holds fourteen work orders, four technicians, and three completed jobs in the history. A critical temperature alert produced a corrective order within seconds, which we verified end to end.

Slide 7. In the demo I use the maintenance board: the automatically created order, assigning a technician, starting and completing it. The takeaway: faults become actionable work automatically, there is one active corrective order per asset, and the history keeps a full record. Yasiru will show how people are notified.

Yasiru

Slide 1. Thank you, Thurunu. I am Yasiru, and I built the Notification Service, which tells people and the console about alerts.

Slide 2. My scope is email and SMS notifications, email templates, notification history, retry handling and the RabbitMQ alert consumer. This covers FR-14 and FR-15.

Slide 3. This is the delivery pipeline. The alert event arrives on a durable RabbitMQ queue, the consumer checks the schema, the message is dispatched on each channel, and the result is stored in the notification database.

Slide 4. The queue has a dead-letter queue for messages that keep failing. There are three channels: email through SMTP, an SMS dispatcher, and an in-app push over STOMP. Templates have full create, read, update and delete. Every message is stored with its status, retry count and reason, and a scheduler retries up to three times, sixty seconds apart, with a manual retry as well.

Slide 5. The service receives the alert event from Monitoring and publishes to two WebSocket topics, one for alerts and one for notifications. Email settings come from environment variables, so credentials never live in the code.

Slide 6. More than seventy-six notifications are recorded, each retried automatically, and the service has fourteen unit test methods. Every attempt is stored, and the in-app push reaches the console instantly.

Slide 7. In the demo I open the notification history, show the delivery status and retry control, and a live toast arriving over WebSocket. The takeaway: every delivery is recorded, in-app push works without email, and consumers are decoupled from Monitoring. Sachinthana will now turn all this data into reports.

Sachinthana

Slide 1. Thank you, Yasiru. I am Sachinthana, and I built the Analytics and Reporting Service.

Slide 2. My scope is dashboard summaries, energy reports, performance reports, maintenance statistics, asset utilisation, KPIs and report export. This covers FR-17.

Slide 3. This is how a report is produced. The service reads data from the Monitoring, Asset and Maintenance services over REST, the calculation service computes the metrics, the results are stored as reports and KPIs, and any report can be exported as CSV.

Slide 4. Energy is the average power of each sensor per hour multiplied by the hours, which gives kilowatt-hours, and it is compared with the expected energy from the rated capacity. Performance reports give availability, efficiency, capacity factor, utilisation and downtime. Maintenance statistics give open orders, the completion rate and the alert count. There are eleven KPI types with targets, and CSV export for reports.

Slide 5. The service uses REST clients only, and never reads another service's database. Aggregation uses the power readings only, and the console charts use the hourly trend that this service provides.

Slide 6. The database holds five energy reports, five performance reports, six KPIs and four dashboards, and the service has eleven unit test methods. Because power is integrated per hour, every figure is true kilowatt-hours.

Slide 7. In the demo I choose a facility and a range, generate an energy report and export it to CSV. The takeaway: reports are computed, stored and exportable, correct units make the figures trustworthy, and executives get read-only access. Finally, we will present what we did together.

All Members

Slide 1. Together, we worked on the parts that make seven independent services behave as one system: testing, documentation, validation, error handling and integration.

Slide 2. Each of us wrote unit tests, documented the API, validated input, handled errors, and coordinated with the other services to keep the contracts compatible. This supports every functional requirement and the non-functional requirements.

Slide 3. Our quality pipeline starts with shared contracts, then service tests with JUnit and Mockito, then gateway checks for authentication and the role matrix, and finally an end-to-end check from a reading to an alert to a work order.

Slide 4. In total there are sixty test methods across seven modules. Documentation includes OpenAPI and Postman collections, SQL schemas, ER diagrams and the integration contract. Validation uses Bean Validation and returns field messages, every service has a global exception handler with consistent JSON, and compatibility comes from UUID references, versioned events and a single gateway.

Slide 5. The integration contract documents the alert event version one. The verified chain is: a reading becomes an alert, which is published to RabbitMQ, and then produces a work order, a notification and a live push. Prometheus scrapes all seven services.

Slide 6. Sixty test methods, seven services behind one gateway, and one versioned event contract. Shared contracts, validation and tests keep seven independent services compatible.

Slide 7. To finish: we agree contracts first and then code, tests protect each service boundary, and every service deploys independently. In the closing demo you will see role-based access, Grafana and RabbitMQ. Thank you. We are happy to take your questions.